

FFT

December 14, 2022

1 Feedback from previous weeks and other and hints

1. Be careful about your environment remembering variables. Make sure your code works in a new *clean* environment. In Colab: `Runtime->restart Runtime`, in Anaconda's Jupyter: `Kernel->Restart`.
2. Keep the file names when saving to GitHub. It's always possible to go back to a previous version, you are not losing anything.
3. Run all the cells before saving to GitHub so the output is saved.
4. Graphs without labels (or units when appropriate) are not worth any point.
5. Do put in sufficient explanatory comments in your code.

For this week you can use these imports at the start of your programs:

```
[ ]: import matplotlib.pyplot as plt
import numpy as np
from numpy.fft import fft, ifft
```

1.1 FFT

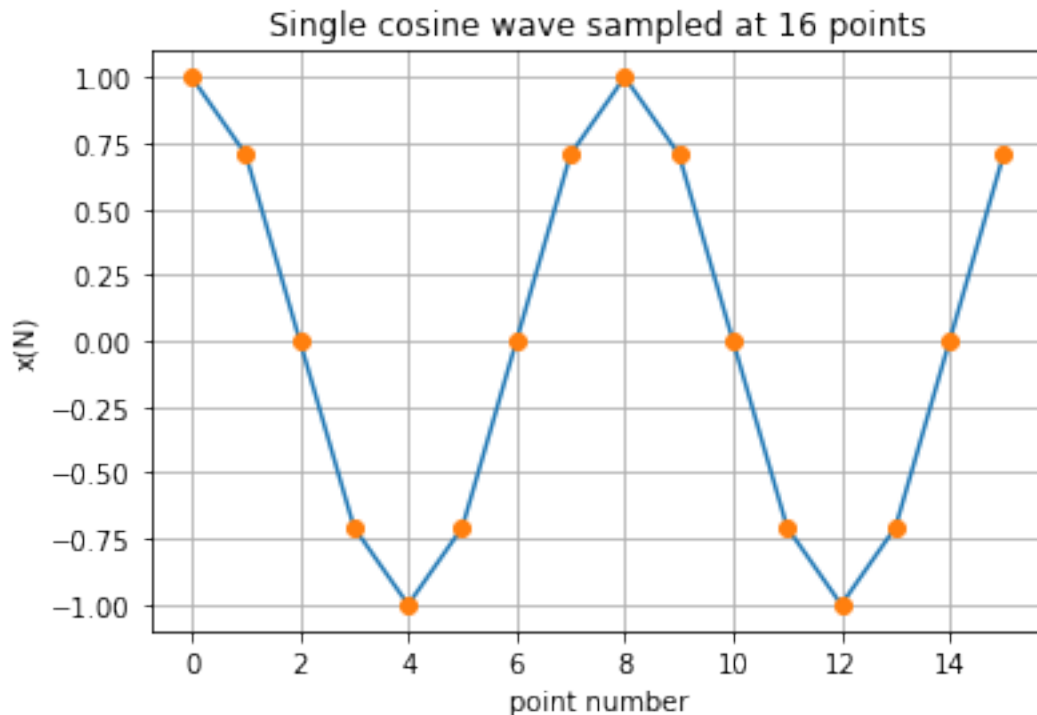
There are several FFT [algorithms](#) which are simple to use. We'll just use `fft` and `ifft`. First set up a data array to test out the method:

```
[ ]: # Examine a small number of points to see what happens more easily
N=16 # the fft algorithm is fastest for N a power of 2
tau = 10.0 # time length
t = np.linspace(0, tau, N, endpoint=False) # creates N points to cover tau
      ↪seconds
# Choose endpoint False so that signal would fit exactly periodically within tau
print(t)
#
f = 0.2 # frequency in Hz
x = np.cos(2*np.pi*f*t) # create x array, amplitude 1.0
plt.plot(x)
plt.plot(x, 'o') # show as points and line
# Note how the plot is exactly periodic

plt.title('Single cosine wave sampled at {} points'.format(N))
plt.xlabel('point number')
```

```
plt.ylabel('x(N)')
plt.grid()
```

```
[0.    0.625 1.25  1.875 2.5   3.125 3.75  4.375 5.    5.625 6.25  6.875
 7.5   8.125 8.75  9.375]
```

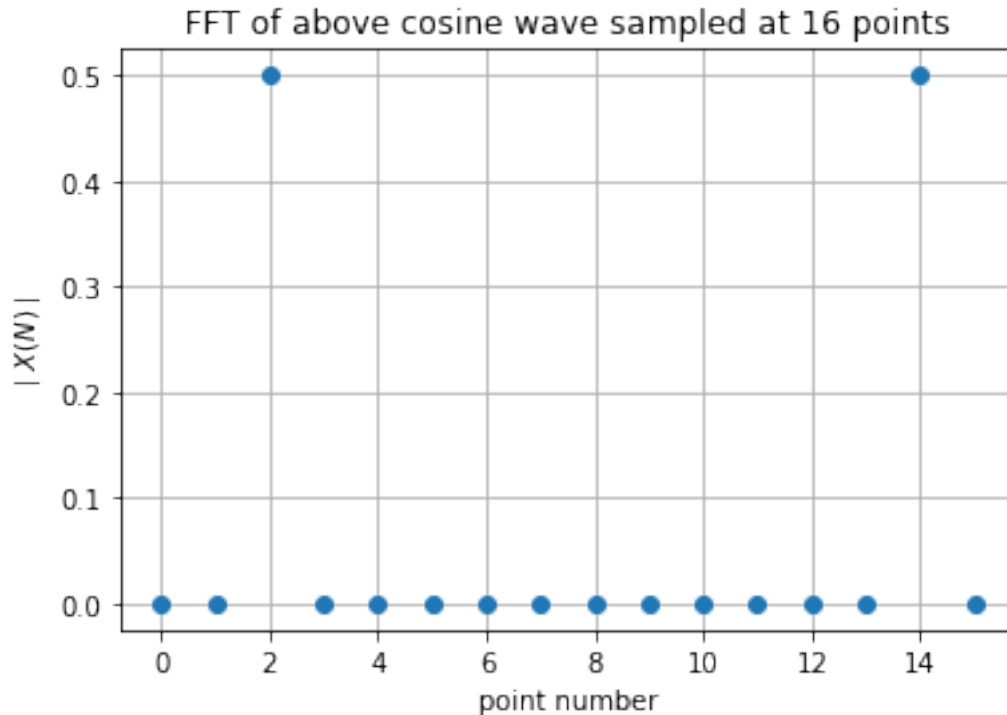


This sets up a cosine wave of just 16 points, so it looks a bit rough when plotted out, but it helps us to see better what happens when we look at the FFT. Use this code snippet to do the FFT:

```
[ ]: X = fft(x)/N # fft and normalise
plt.plot(np.abs(X), 'o')
plt.title('FFT of above cosine wave sampled at '+str(N)+' points')
plt.xlabel('point number')
plt.ylabel('$\mid X(N)\mid$')
plt.grid()
print(X)
```

```
[ 1.24474906e-17+0.00000000e+00j -8.73114191e-17+5.19299245e-17j
 5.00000000e-01-1.72254642e-16j  1.69824100e-17-7.40843115e-17j
 1.53080850e-17-2.77555756e-17j  2.63153929e-17-7.40843115e-17j
 5.55111512e-17-4.97899625e-17j  4.40136163e-17+5.19299245e-17j
-4.30636606e-17+0.00000000e+00j  4.40136163e-17-5.19299245e-17j
 5.55111512e-17+4.97899625e-17j  2.63153929e-17+7.40843115e-17j
 1.53080850e-17+2.77555756e-17j  1.69824100e-17+7.40843115e-17j]
```

5.00000000e-01+1.72254642e-16j -8.73114191e-17-5.19299245e-17j]



Two things to notice:

1. Point 0 is the zero frequency component, points 1 to 8 are the positive frequency components, points 9 to 15 are the negative frequency components. To put the latter in the ‘correct’ position we can reorder them, so that point 15 appears effectively in the -1 position, point 14 in the -2 position, and so on (since the FT is periodic with period F_s , there isn’t really a ‘correct’ interval any more. The algorithm returns the interval 0 to F_s , but you might prefer $-F_s/2$ to $F_s/2$. You will meet exactly this idea in describing periodic crystal structures, where the FT is actually the reciprocal lattice).

But since the negative frequency components just repeat the positive ones (as the array x is real) we can just ignore them.

2. Printing X shows that it has real and imaginary parts, but that the imaginary part is zero within numerical error. This is because we picked a cosine wave as the test data.

The *wrapper* function to the fft below is handy as it returns not just the Fourier amplitudes but also the corresponding frequencies. (There are actually other ways to do it using numpy directly, but it helps to show these details.)

```
[ ]: def Spectrum1(x, Fs):  
    '''  
    One-sided amplitude spectrum of x(t).  
    '''
```

```

N = len(x) # length of the signal
frq = (float(Fs)/N)*np.arange(0, N/2+1) # one-sided frequency range
X = fft(x)/N # fft computing and normalization
X = X[0:N//2+1] # take just positive values
# (in python3, // ensures integer division)

# frq is array of actual frequencies
# X is complex array of amplitudes
return frq , X

```

The division by N normalisation is there because the `fft` is implemented by numpy with a different pre-factor, equation (11); putting it in gives agreement with the ordinary Fourier transform amplitudes defined earlier. (There is no uniform convention on defining the prefactors.) Later, we'll see that we have to multiply by N again before doing the inverse discrete Fourier transform `ifft`.

2 Exercises

Mark weighting is in brackets. **Do add comments to your code, you'll lose points if your code is hard to understand.** Graphs without labels (or units when appropriate) are not worth any point.

Read section Implementation above and do the following exercises.

(1). [2] Plot the original signal x against time instead of point number. (Use $N=64$ points.)

```

[ ]: # number of points
N=64 # the fft algorithm is fastest for N a power of 2
# [0.5] to set N=64

tau = 10.0 # time length
t = np.linspace(0, tau, N, endpoint=False) # creates N points to cover tau
      ↪seconds
# Choose endpoint False so that signal would fit exactly periodically within tau

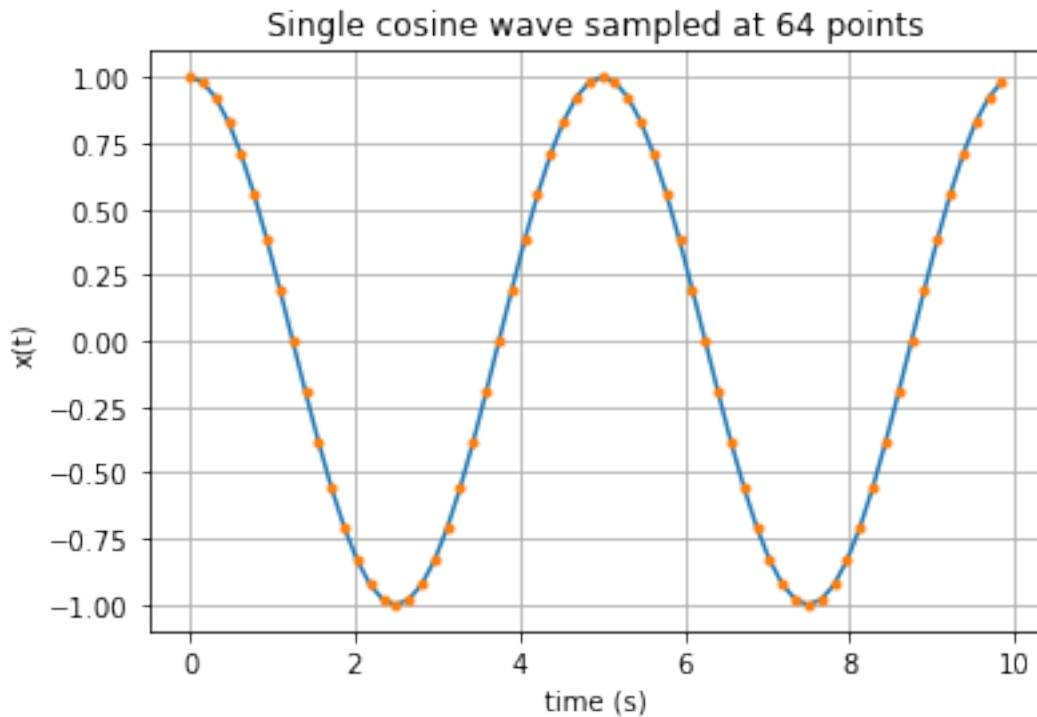
f = 0.2 # frequency in Hz
x = np.cos(2*np.pi*f*t) # create x array, amplitude 1.0

# plotting x as a function of time
plt.plot(t, x)
plt.plot(t, x, '.') # show as points and line
# Note how the plot is exactly periodic
# [0.5] to make the plot against time

plt.title('Single cosine wave sampled at {} points'.format(N))
plt.xlabel('time (s)')
plt.ylabel('x(t)')
plt.grid()
# [1] for the plot, including the x-label

```

```
# (both lines and points are not necessary).
```

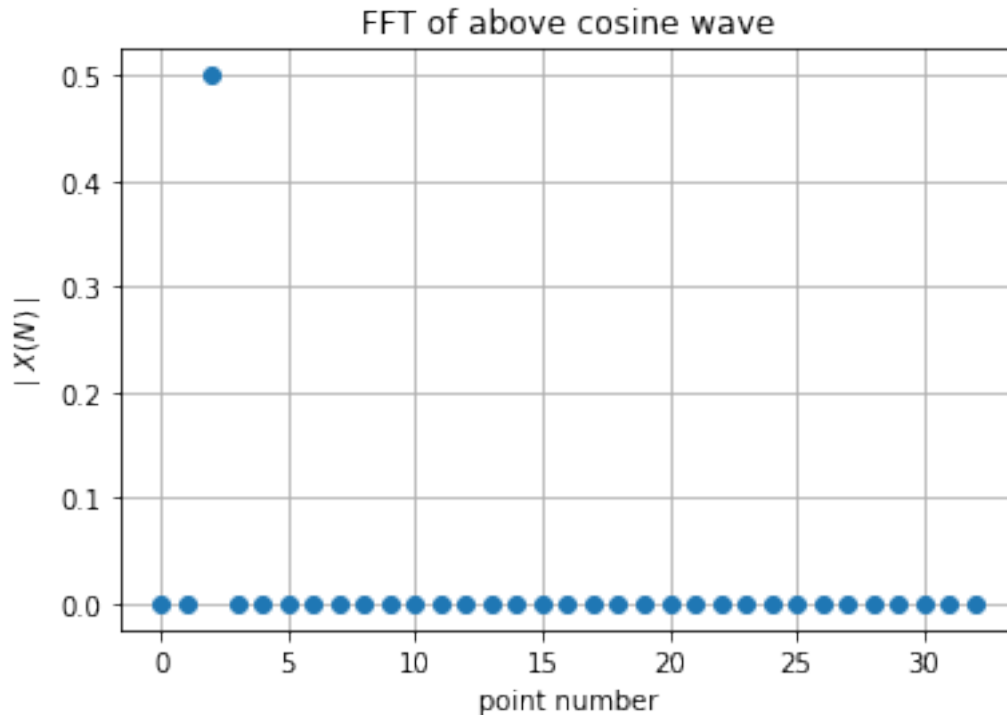


(2). [2] Plot just the positive frequencies of the FFT using the `fft` routine of `numpy`. This is a plot versus point number.

```
[ ]: X = fft(x)/N # fft and normalise

# selecting the first half of the X array
# to get only the positive frequencies
plt.plot(np.abs(X[0:N//2+1]), 'o')
# [1] to select the positive frequencies

plt.title('FFT of above cosine wave')
plt.xlabel('point number')
plt.ylabel('$\mid X(N)\mid$')
plt.grid()
# [1] for the plot. Other representations of the complex
# number X are fine (here I only plot the amplitude),
# as long as they are clear.
```



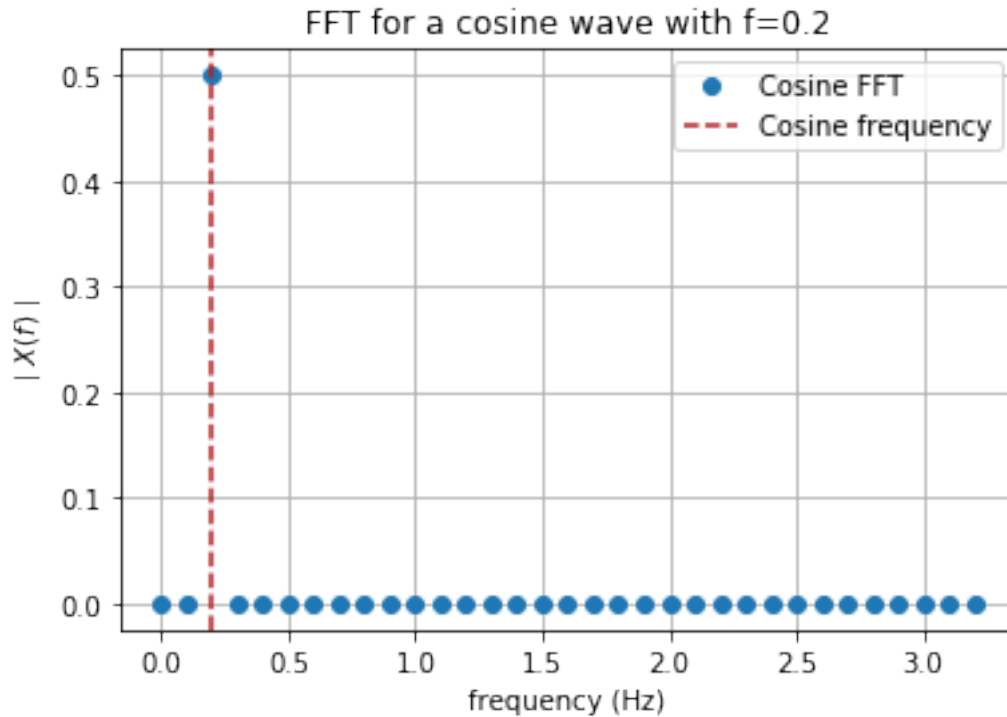
(3). [2] For this question **and all the following**, use the `Spectrum1` function. Calculate and plot the FFT of x versus frequency. (**In the rest of the exercises just plot against frequency.**) Why is the signal amplitude you find equal to 0.5 instead of 1?

Recall: The sampling rate is $F_s = 1/\Delta t = N/\tau$. Read the section Summary above!

```
[ ]: # Generating the FFT of x:
frq, X = Spectrum1(x, Fs=N/tau)
# [1] for the call to Spectrum1(), including the right value for Fs

# plotting the result:
plt.plot(frq, np.abs(X), 'o', label='Cosine FFT')
plt.axvline(f, ls='--', color='firebrick', label='Cosine frequency')
plt.title('FFT for a cosine wave with f={}'.format(f))
plt.xlabel('frequency (Hz)')
plt.ylabel('$\mid X(f)\mid$')
plt.legend()
plt.grid()
# [1] for the plot. Other representations of the complex
# number X are fine (here I only plot the amplitude),
# as long as they are clear.

# The magnitude is only 0.5 because we only selected the positive frequencies
```

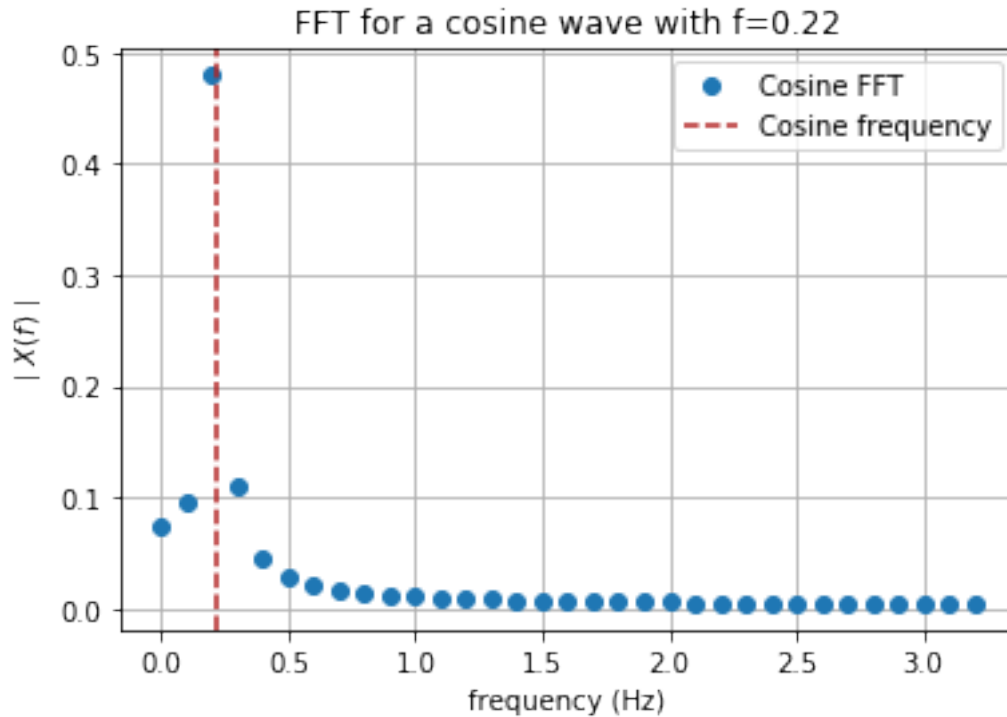


(4). [2] Show what happens if you change the frequency of the original data to 0.22 Hz instead of 0.2 Hz. *Explain what you see to the demonstrator.* (Hint: It helps to plot both the real and imaginary parts of X , and to plot $|X|$.)

```
[ ]: # Setting up the data
f = 0.22 # frequency in Hz
x = np.cos(2*np.pi*f*t) # create x array, amplitude 1.0
```

```
[ ]: # Generating the FFT of x:
frq, X = Spectrum1(x, Fs=N/tau)

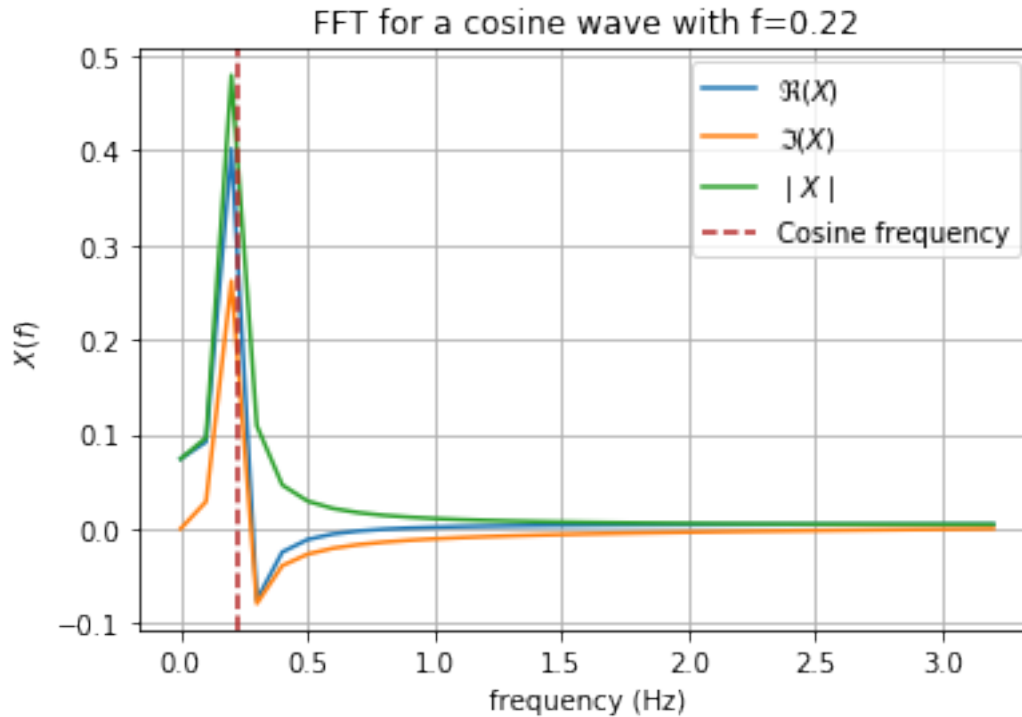
# plotting the results using |X(f)|
plt.plot(frq, np.abs(X), 'o', label='Cosine FFT')
plt.axvline(f, ls='--', color='firebrick', label='Cosine frequency')
plt.title('FFT for a cosine wave with f={}'.format(f))
plt.xlabel('frequency (Hz)')
plt.ylabel('$|X(f)|$')
plt.legend()
plt.grid()
```



```
[ ]: # plotting the results using Re(X), Im(X) and |X(f)|
plt.plot(frq,np.real(X),label='$\text{Re}(X)$')
plt.plot(frq,np.imag(X),label='$\text{Im}(X)$')
plt.plot(frq,np.abs(X),label='$\mid X \mid$')
plt.axvline(f,ls='--',color='firebrick',label='Cosine frequency')
plt.title('FFT for a cosine wave with f={}'.format(f))
plt.xlabel('frequency (Hz)')
plt.ylabel('$|X(f)|$')
plt.legend()
plt.grid()

# [1] for the plot(s), [1] for the explanation
# which plot(s) is up to the students, as long
# as the explanation makes sense...

# In this example our DFT does not have a point at 0.22.
# The maximum is not quite at 0.5, but if we 'interpolate-by-eye',
# we can see that the peak would be at (0.22,0.5).
```

(5). [4] (a) Investigate what happens as you sample the original data with more points, and (b) what happens as you increase the length τ of the original data (for both 0.2 and 0.22 Hz).

```
[ ]: # setting up a fancy multiplot to vary N (columns) and \tau (rows)
fig, ax = plt.subplots(3, 3, sharey=True, figsize=(12,8))

# Looping over N (columns)
for i,N in enumerate([64,128,512]):
    # Looping over \tau (rows)
    for j,tau in enumerate([10,20,40]):
        # Now that N and \tau are fixed, compute the time array
        t = np.linspace(0, tau, N, endpoint=False)

        # Loop over the two frequencies:
        for f in [0.2,0.22]:

            # compute the x data
            x = np.cos(2*np.pi*f*t)

            # do the FFT
            frq, X = Spectrum1(x, Fs=N/tau)

            # and plot this specific N,\tau combination
            ax[i, j].plot(frq, np.abs(X),label='f={}'.format(f))
```

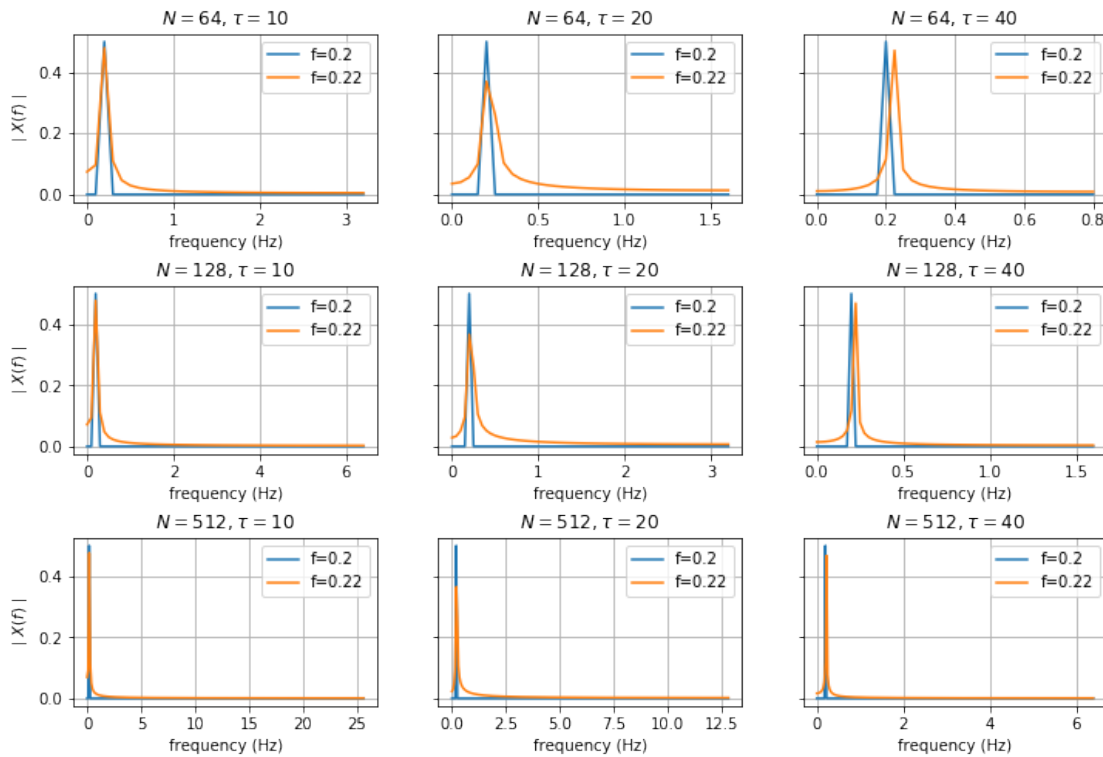
```

ax[i, j].set_title(r'$N={}\$, \tau ={}\$'.format(N,tau))
ax[i, j].set_xlabel('frequency (Hz)')
ax[i, j].legend()
ax[i, j].grid(True)

# We only need to specify the y_label for the first column
ax[i, 0].set_ylabel('$\mid X(f)\mid$')

# Adjust the size so the x_label fits
plt.subplots_adjust(hspace=0.5)

```



```

[ ]: # Now, the exact same thing, but we will fix (and share) the x-axis
      # so that all plots have the same range of frequency values

```

```

fig, ax = plt.subplots(3, 3, sharey=True, sharex=True, figsize=(12,8))

# Looping over N (columns)
for i,N in enumerate([64,128,512]):
    # Looping over \tau (rows)
    for j,tau in enumerate([10,20,40]):
        # Now that N and \tau are fixed, compute the time array
        t = np.linspace(0, tau, N, endpoint=False)

```

```

# Loop over the two frequencies:
for f in [0.2,0.22]:

    # compute the x data
    x = np.cos(2*np.pi*f*t)

    # do the FFT
    frq, X = Spectrum1(x, Fs=N/tau)

    # and plot this specific N,\tau combination
    ax[i, j].plot(frq, np.abs(X),label='f={}'.format(f))
    ax[i, j].set_title(r'$N={}$, \tau ={}$'.format(N,tau))
    ax[i, j].legend()
    ax[i, j].grid(True)

    # Here we only need to specify the x_label for the last row
    ax[-1, j].set_xlabel('frequency (Hz)')

# We only need to specify the y_label for the first column
ax[i, 0].set_ylabel('$\mid X(f)\mid$')

# Set the x-axis range
plt.xlim(0,2);

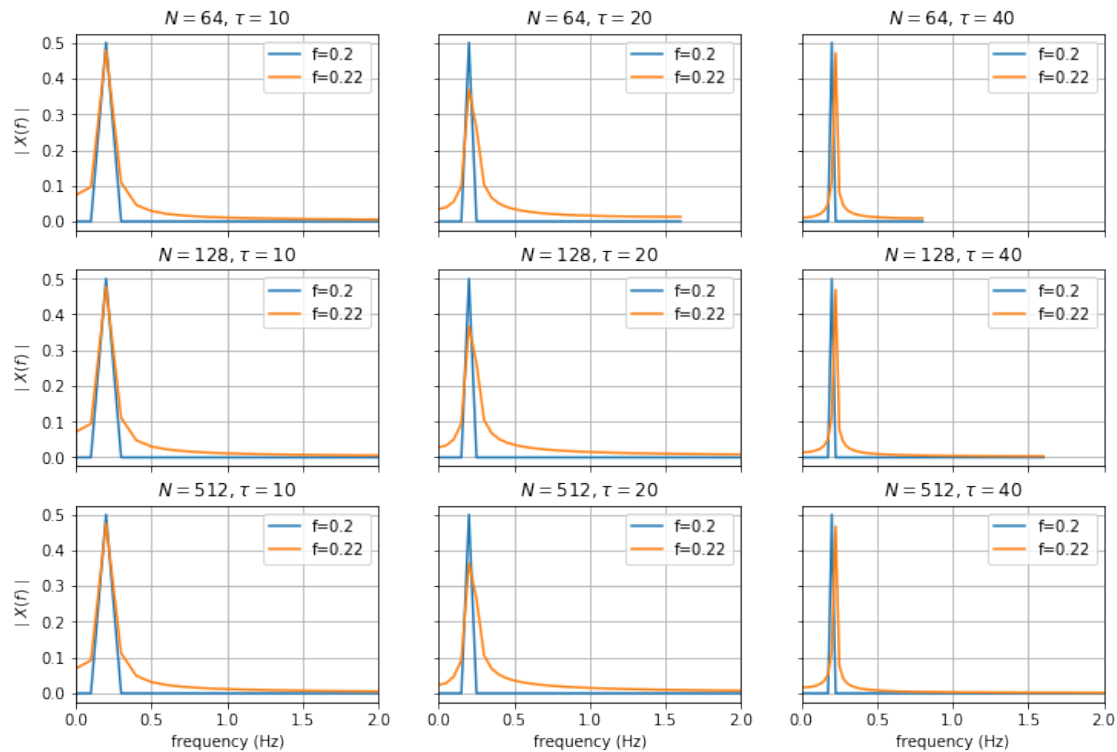
# [1] for a plot that shows what happens as N increases
# [1] for the explanation (we can see higher frequencies)

# [1] for a plot that shows what happens as \tau increases
# [1] for the explanation (we can see finer frequencies)

# Any combination of plot and explanation that is clear works.

# Increasing \tau increases the resolution
# Increasing N increases the maximum frequency

```



(6). [4] Describe what happens to the FFT if you choose non-zero phase in the cosine waves (for both 0.2 and 0.22 Hz). (Hint: Plot both the real and imaginary parts of X in one graph, and $|X|$ and the argument $\arg(X)$ as a function of frequency on another graph. The function `np.angle()` will be helpful to do this.)

```
[ ]: N = 64
tau = 10.0
t = np.linspace(0, tau, N, endpoint=False) # creates N points to cover tau
      ↪ seconds
# Choose endpoint False so that signal would fit exactly periodically within tau

f = 0.22 # frequency in Hz
phase = np.pi/2.5 # phase in radian

x = np.cos(2*np.pi*f*t+phase) # create x array, amplitude 1.0
# [1] to compute x with a phase

frq, X = Spectrum1(x, Fs=N/tau)
# [1] to do the FFT

fig, ax = plt.subplots(2, 1, sharex=True, sharey=True)

ax[0].plot(frq, np.real(X), label=r'$\text{Re}(X)$')
```

```

ax[0].plot(frq,np.imag(X),label=r'$\Im(X)$')
ax[0].set_ylabel(r'$X(f)$')
ax[0].legend()
ax[0].grid(True)

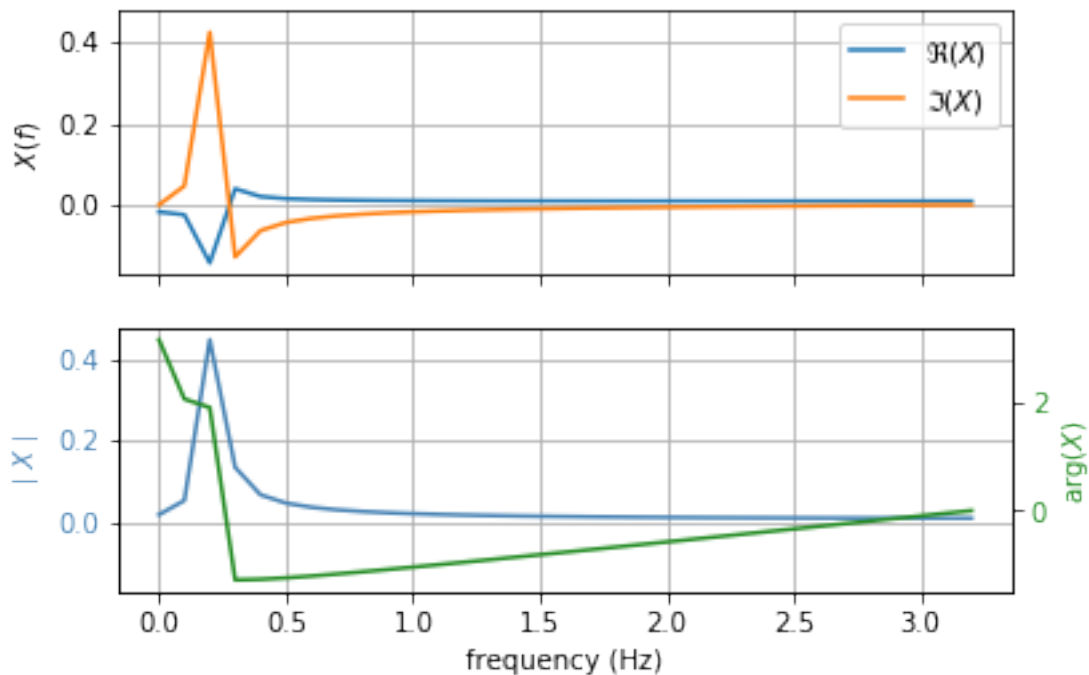
ax[1].plot(frq,np.abs(X),label=r'$\mid X \mid$',color='steelblue')
ax[1].tick_params(axis='y', labelcolor='steelblue')
ax[1].set_ylabel(r'$\mid X \mid$', color='steelblue')
ax[1].set_xlabel('frequency (Hz)')
ax[1].grid(True)

ax2 = ax[1].twinx()
ax2.plot(frq,np.angle(X),color='forestgreen')
ax2.tick_params(axis='y', labelcolor='forestgreen')
ax2.set_ylabel(r'$\arg(X)$', color='forestgreen')

plt.subplots_adjust(wspace=0.4)
# [1] for the plot(s)
# [1] for the explanation

# Only the phase of  $X(f)$  changes

```



(7). [4] Write a function `TestData(N, Fs)` which creates N points of data containing three cosine waves with amplitudes and frequencies $a_1 = 1.0, f_1 = 12.3, a_2 = 0.5, f_2 = 7.5, a_3 = 0.2, f_3 = 30.0$. Use this function to plot the FFT for $N=512, 1024$ and 2048 and explain the results.

Recall: The time separation between points is $\Delta t = 1/F_s$. Read the section Summary above!

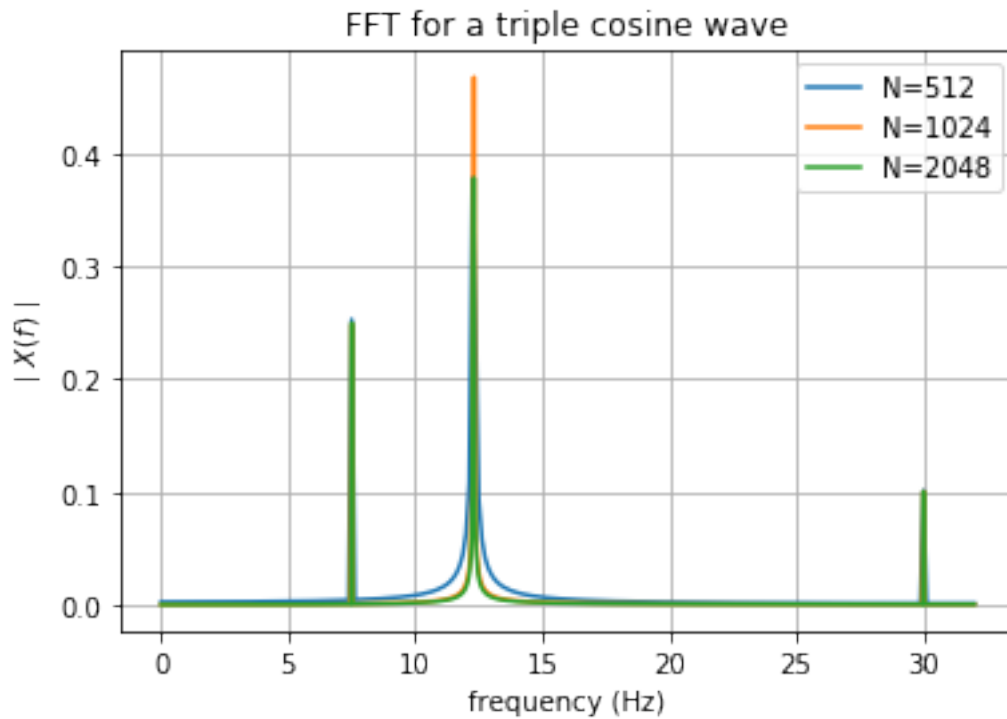
```
[ ]: def TestData(N, Fs):  
    '''  
    Creates N points from the sum of cosines sampled at  
    frequency Fs. The cosines have amplitudes and frequencies:  
  
    amps = [1.0,0.5,0.2]  
    freqs = [12.3,7.5,30.0]  
    '''  
    tau = N/Fs # time length  
    t = np.linspace(0, tau, N, endpoint=False) # creates N points to cover tau  
    ↪seconds  
  
    freqs = [12.3,7.5,30.0] # frequencies in Hz  
    amps = [1.0,0.5,0.2] # amplitudes (dimentionless)  
  
    # set-up the return array x to have the same dimentions as t  
    x = np.zeros_like(t)  
    # loop over the amplitudes and frequencies  
    for a,f in zip(amps,freqs):  
        # add the cosines  
        x += a*np.cos(2*np.pi*f*t)  
  
    return x  
# [2] for a nice function
```

```
[ ]: # Sampling rate needs to be at least twice the maximum frequency  
# of the signal of interest to avoid aliasing:  
Fs = 64  
  
# looping over the number of points N  
for N in [512,1024,2048]:  
    # creating the signal  
    x = TestData(N, Fs)  
    # computing the FFT  
    frq, X = Spectrum1(x, Fs)  
    # and plot the result  
    plt.plot(frq,np.abs(X),label='N={}'.format(N))  
  
plt.title('FFT for a triple cosine wave')  
plt.xlabel('frequency (Hz)')  
plt.ylabel('$\mid X(f)\mid$')  
plt.legend()  
plt.grid()  
# [1] for the plot  
# [1] for the explanation (if Fs<2*30.0, the explanation should
```

```
# include the concepts of "aliasing" or "leaking")
```

```
# If we do have a high enough  $F_s$ , we recover the three peaks
```

```
# corresponding to the three cosines' frequencies.
```



```
[ ]:
```